

Nome: Leandro Menali Ferreira

Projeto: Case – Looqbox

Teste SQL

Após acessar nosso banco de dados, crie consultas usando o esquema lookbox_challenge para responder às seguintes perguntas:

1. Quais são os 10 produtos mais caros da empresa?

Query criada para a pesquisa (arquivo: SQL_1.sql):

```
SELECT PRODUCT_NAME, PRODUCT_VAL
FROM data_product
ORDER BY PRODUCT_VAL DESC LIMIT 10;
```

Resultado:

	AZ PRODUCT_NAME	123 PRODUCT_VAL
1	Whisky Escoces THE MACALLAN Ruby Garrafa 700ml com Caixa	741.99
2	Whisky Escoces JOHNNIE WALKER Blue Label Garrafa 750ml	735.9
3	Cafeteira Expresso 3 CORACOES Tres Modo Vermelho	499
4	Vinho Portugues Tinto Vintage QUINTA DO CRASTO Garrafa 750ml	445.9
5	Escova Dental Eletrica ORAL B D34 Professional Care 5000 110v	399.9
6	Champagne Rose VEUVE CLICQUOT PONSARDIM Garrafa 750ml	366.9
7	Champagne Frances Brut Imperial MOET Rose Garrafa 750ml	359.9
8	Conjunto de Panelas Allegra em Inox TRAMONTINA 5 Pecas Gratis Utensilios 5 Pecas	359
9	Whisky Escoces CHIVAS REGAL 18 Anos Garrafa 750ml	329.9
10	Champagne Frances Brut Imperial MOET & CHANDON Garrafa 750ml	315.9

2. Quais são as seções dos departamentos 'BEBIDAS' e 'PADARIA'?

Query criada para a pesquisa (arquivo: SQL_2.sql):

```
SELECT DISTINCT DEP_NAME, SECTION_NAME
FROM data_product
WHERE DEP_NAME IN ('BEBIDAS', 'PADARIA')
ORDER BY DEP_NAME, SECTION_NAME ASC;
```

Resultado:

	A-Z DEP_NAME	A-Z SECTION_NAME
1	BEBIDAS	BEBIDAS
2	BEBIDAS	CERVEJAS
3	BEBIDAS	REFRESCOS
4	BEBIDAS	VINHOS
5	PADARIA	DOCES-E-SOBREMESAS
6	PADARIA	GESTANTE
7	PADARIA	PADARIA
8	PADARIA	QUEIJOS-E-FRIOS

3. Qual foi o total de vendas de produtos (em dólares) de cada área de negócios no primeiro trimestre de 2019?

Query utilizada para a pesquisa (arquivo: SQL_3.sql):

```
SELECT c.BUSINESS_NAME, SUM(s.SALES_VALUE) as TOTAL_SALES_VALUE
FROM data_store_cad c JOIN data_store_sales s
ON c.STORE_CODE = s.STORE_CODE
WHERE s.DATE BETWEEN '2019-01-01' AND '2019-03-31'
GROUP BY c.BUSINESS_NAME
ORDER BY TOTAL_SALES_VALUE DESC;
```

Resultado:

	A-Z BUSINESS_NAME	123 TOTAL_SALES_VALUE
1	Farma	81,776,691.73
2	Varejo	81,032,347.65
3	Atacado	80,384,884.6
4	Proximidade	80,171,122.8
5	Posto	32,072,326.4

Teste Python

1) A equipe de desenvolvimento estava cansada de desenvolver as mesmas consultas de sempre, variando apenas os filtros de acordo com as exigências do chefe.

Como novo membro da equipe, sua missão agora é criar uma função dinâmica em Python, da maneira mais flexível possível, para gerar consultas e recuperar um dataframe com base em três parâmetros:

- código_do_produto: inteiro
- store_code: inteiro
- data: lista de strings semelhantes a ISO
- Data, por exemplo
 - ['2019-01-01', '2019-01-31']

Deverá ter o seguinte aspeto: `my_data = retrieve_data(product_code, store_code, date)`

Resposta:

O uso de IA/LLM foi utilizado para otimizar o script, colocando validações, como o caso das datas inseridas, para que em uma eventual inserção de uma data inválida, o script aponte o erro (arquivo: python1.py).

```
import mysql.connector
import pandas as pd
from datetime import datetime

def validate_date(date_str):
    try:
        datetime.strptime(date_str, "%Y-%m-%d")
        return True
    except ValueError:
        return False

def retrieve_data(product_code:int, store_code:int, date:list):
    if not isinstance(date, list) or len(date) != 2:
        raise ValueError("date must be a list with [start_date, end_date]")
```

```

start_date, end_date = date
if start_date > end_date:
    raise ValueError("start_date must be <= end_date")

if not validate_date(start_date) or not validate_date(end_date):
    raise ValueError("Invalid date format. Use YYYY-MM-DD")
#conectar com o banco de dados
try:
    conn = mysql.connector.connect(
        host="35.199.115.174",
        user="loobox-challenge",
        password="loox-challenge"
    )
    cursor = conn.cursor()
    cursor.execute("USE `loobox-challenge`")
    """

    cursor = conn.cursor()
    cursor.execute("SHOW DATABASES;")

    for db in cursor:
        print(db)
    """

    query = """
        SELECT *
        FROM data_product_sales
        WHERE PRODUCT_CODE = %s
        AND STORE_CODE = %s
        AND DATE BETWEEN %s AND %s
    """

    params = (product_code, store_code, start_date, end_date)

    df = pd.read_sql(query, conn, params=params)

    conn.close()
    return df

finally:
    if conn:
        conn.close()

```

```

PRODUCT_CODE = 18
STORE_CODE = 1
START_DATE = '2019-01-01'
END_DATE = '2019-01-31'

```

```
my_data = retrieve_data(
```

```

    product_code=PRODUCT_CODE,
    store_code=STORE_CODE,
    date=[START_DATE, END_DATE]
)
print(my_data.head(10))

```

Saída do script:

	STORE_CODE	PRODUCT_CODE	DATE	SALES_VALUE	SALES_QTY
0	1	18	2019-01-01	708.5	65.0
1	1	18	2019-01-02	1297.1	119.0
2	1	18	2019-01-03	1144.5	105.0
3	1	18	2019-01-04	1090.0	100.0
4	1	18	2019-01-05	893.8	82.0
5	1	18	2019-01-06	741.2	68.0
6	1	18	2019-01-07	654.0	60.0
7	1	18	2019-01-08	741.2	68.0
8	1	18	2019-01-09	1373.4	126.0
9	1	18	2019-01-10	1068.2	98.0

2) Um novo cliente lhe enviou duas consultas prontas para serem respondidas.

Além disso, ele lhe deu este conjunto de instruções:

- Utilize as consultas tal como estão (não as modifique nem crie uma nova);
- Por favor, filtre o período dentro do intervalo especificado:
 - ['2019-10-01','2019-12-31']

Resposta:

Utilização de ajuda pontual no df_grouped para criação do TM (arquivo: python2.py).

```

import mysql.connector
import pandas as pd
import numpy as np

conn = mysql.connector.connect(
    host="35.199.115.174",
    user="loobox-challenge",
    password="loox-challenge"
)
cursor = conn.cursor()
cursor.execute("USE `loobox-challenge`")

# -----
# QUERY 1 -> STORE_NAME
# -----

```

```

q1 = """
SELECT
    STORE_CODE,
    STORE_NAME,
    START_DATE,
    END_DATE,
    BUSINESS_NAME,
    BUSINESS_CODE
FROM data_store_cad
"""

df_stores = pd.read_sql(q1, conn)

# -----
# QUERY 2 -> SALES
# -----
q2 = """
SELECT
    STORE_CODE,
    DATE,
    SALES_VALUE,
    SALES_QTY
FROM data_store_sales
WHERE DATE BETWEEN '2019-01-01' AND '2019-12-31'
"""

df_sales = pd.read_sql(q2, conn)
conn.close()
df_sales["DATE"] = pd.to_datetime(df_sales["DATE"])
df_sales = df_sales[
    (df_sales["DATE"] >= "2019-10-01") &
    (df_sales["DATE"] <= "2019-12-31")
]

df = df_sales.merge(df_stores, on="STORE_CODE", how="left")
df_grouped = df.groupby(["STORE_NAME", "BUSINESS_NAME"]).agg(
    total_sales=("SALES_VALUE", "sum"),
    total_qty=("SALES_QTY", "sum")
).reset_index()

df_grouped["TM"] =
np.round(df_grouped["total_sales"]/df_grouped["total_qty"],2)

final_df = df_grouped[[
    "STORE_NAME",
    "BUSINESS_NAME",
    "TM"
]]

final_df.columns = ["Loja", "Categoria", "TM"]

```

```
final_df = final_df.sort_values(by="Loja", ascending=True)
print(final_df)
```

Saída do Script:

	Loja	Categoria	TM
0	Bahia	Atacado	15.39
1	Bangkok	Posto	13.67
2	Belem	Proximidade	15.37
3	Berlin	Proximidade	15.39
4	Buenos Aires	Atacado	15.39
5	Chicago	Varejo	15.53
6	Dubai	Atacado	15.39
7	Hong Kong	Farma	26.35
8	London	Farma	28.99
9	Madri	Farma	29.03
10	Miami	Posto	13.67
11	New York	Proximidade	15.39
12	Paris	Proximidade	15.39
13	Rio de Janeiro	Farma	29.59
14	Roma	Varejo	15.39
15	Salvador	Atacado	15.39
16	Sao Paulo	Varejo	15.39
17	Sidney	Posto	13.67
18	Tokio	Varejo	15.39
19	Vancouver	Posto	13.67

3) Criando sua própria visualização

Crie pelo menos um gráfico usando a tabela IMDB_movies . O código deve ser em Python e você pode usar quaisquer bibliotecas, dados da tabela e formato gráfico que desejar. Explique por que você escolheu a(s) visualização(ões) que está enviando.

Resposta:

(arquivo: python3.py)

Primeiro me conectei a tabela do banco de dados:

```
import mysql.connector
import pandas as pd
import matplotlib.pyplot as plt

conn = mysql.connector.connect(
    host="35.199.115.174",
    user="looqbox-challenge",
    password="looq-challenge",
    database="looqbox-challenge"
)

query = """
SELECT
    Title,
```

```

Genre,
Director,
Actors,
Year,
Runtime,
Rating,
Votes,
RevenueMillions,
Metascore
FROM IMDB_movies
"""

df = pd.read_sql(query, conn)

conn.close()
display(df)

```

Posteriormente, foi realizada a visualização da quantidade de filmes produzidos por ano, com destaque para os anos em que houve redução no total de produções em comparação ao ano anterior. Durante o desenvolvimento, também foi aplicado um tratamento para que o primeiro ano da série temporal não fosse considerado na comparação, evitando inconsistências visuais decorrentes da ausência de um período anterior de referência.

```

df_count = df.groupby("Year").size().reset_index(name="Movies_Count")
df_count = df_count.sort_values("Year")
df_count["diff"] = df_count["Movies_Count"].diff()
colors = df_count["diff"].apply(
    lambda x: "green" if pd.isna(x) else ("green" if x >= 0 else "red")
)

plt.figure(figsize=(10,6))

plt.bar(df_count["Year"], df_count["Movies_Count"], color=colors)

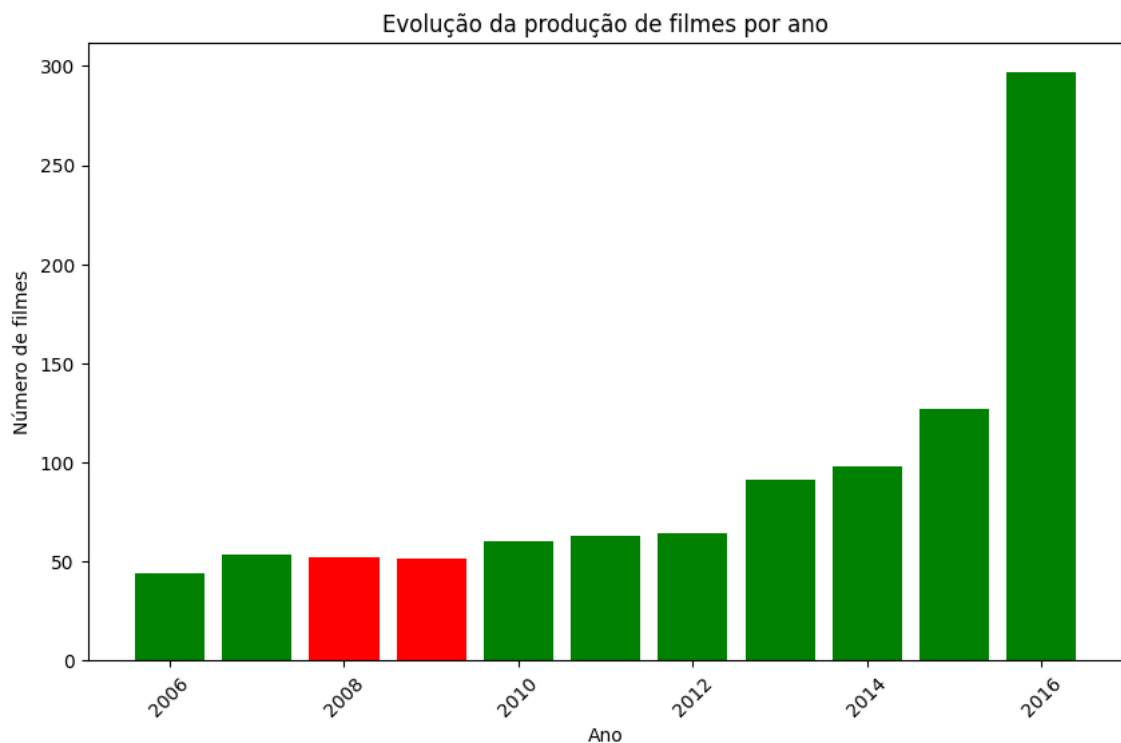
plt.title("Evolução da produção de filmes por ano")
plt.xlabel("Ano")
plt.ylabel("Número de filmes")

plt.xticks(rotation=45)

plt.show()

```


Foi escolhida a visualização de barras para representar a quantidade de filmes lançados por ano, permitindo uma análise clara da evolução da produção cinematográfica ao longo do tempo. Esse tipo de gráfico facilita a comparação entre diferentes períodos, destacando anos com maior ou menor volume de lançamentos. Além disso, foram utilizadas cores distintas para evidenciar os anos em que houve redução na quantidade de filmes produzidos em relação ao ano anterior, tornando a interpretação visual mais intuitiva e dinâmica.



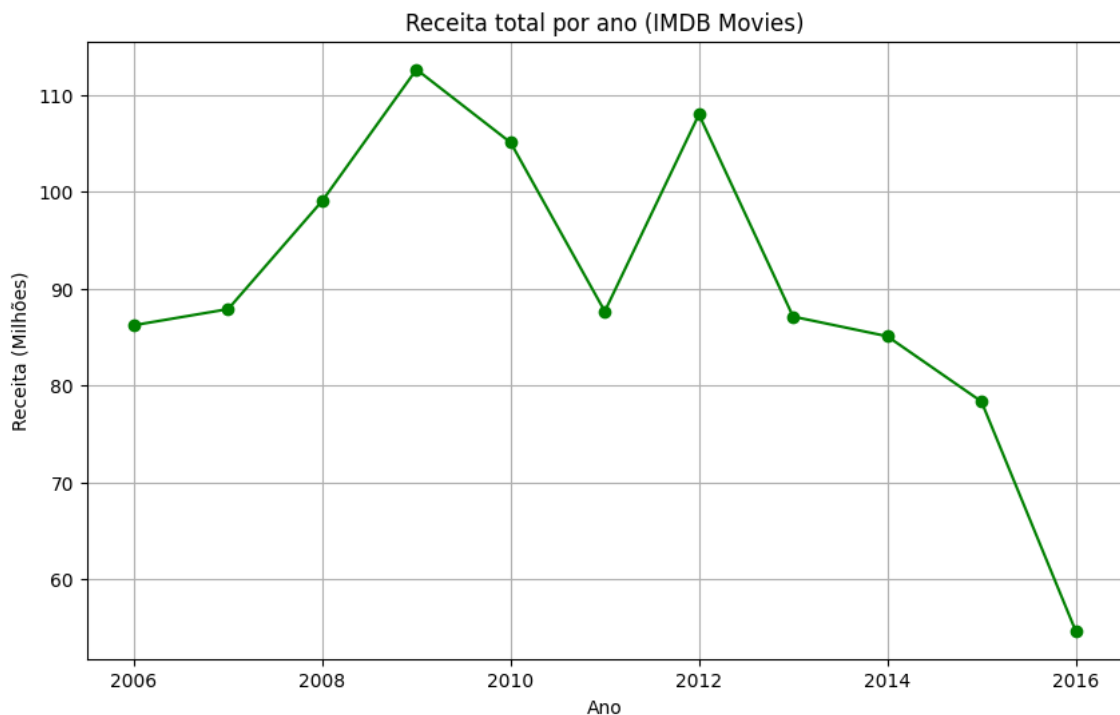
Em seguida, foi realizada uma análise da receita média dos filmes ao longo dos anos. Essa visualização permite compreender o desempenho financeiro médio das produções cinematográficas em cada período, possibilitando identificar tendências de valorização ou redução do retorno comercial dos filmes. A análise também auxilia na interpretação sobre a qualidade e o potencial de mercado das produções lançadas ao longo do tempo, considerando que variações na receita média podem indicar mudanças no perfil da indústria cinematográfica e no impacto comercial das obras produzidas.

```
df_year = df.groupby("Year")["RevenueMillions"].mean().reset_index()
df_year = df_year.sort_values("Year")
plt.figure(figsize=(10,6))
plt.plot(df_year["Year"], df_year["RevenueMillions"], marker="o",
color="g")
```

```
plt.title("Receita total por ano (IMDB Movies)")
plt.xlabel("Ano")
plt.ylabel("Receita (Milhões)")
plt.grid(True)
plt.show()
```

Foi escolhida uma visualização de linha para analisar a evolução da receita média dos filmes ao longo dos anos. Esse tipo de gráfico é adequado para identificar tendências temporais, permitindo observar variações de crescimento ou queda no desempenho financeiro da indústria cinematográfica ao longo do período analisado.

A agregação por ano facilita a comparação entre diferentes períodos, reduzindo o impacto de variações individuais e destacando o comportamento geral da receita média ao longo do tempo. Essa abordagem permite compreender, em uma perspectiva mais ampla, possíveis mudanças no perfil comercial das produções cinematográficas e no potencial de arrecadação dos filmes lançados em cada período.



Seguindo a linha de raciocínio, posteriormente foi desenvolvido um gráfico para analisar os filmes com maiores receitas da base de dados, juntamente com seus

respectivos anos de lançamento. Durante o desenvolvimento, também foi realizada uma implementação complementar para associar o nome de cada filme ao ano em que foi lançado, enriquecendo a visualização e facilitando a contextualização temporal das produções com maior desempenho financeiro. Ajuda pontual para inserção do nome do filme associado com o ano de lançamento.

```
df["RevenueMillions"] = pd.to_numeric(df["RevenueMillions"],
errors="coerce")

df = df.dropna(subset=["RevenueMillions", "Title", "Year"])
top10 = df.sort_values("RevenueMillions",
ascending=False).head(10).copy()
top10["label"] = top10["Title"] + " (" +
top10["Year"].astype(int).astype(str) + ")"

plt.figure(figsize=(10,6))
bars = plt.barh(
    top10["label"],
    top10["RevenueMillions"],
    color="green"
)
plt.title("Top 10 filmes por receita (em milhões)")
plt.xlabel("Receita (Milhões)")
plt.ylabel("Filme (Ano)")

plt.gca().invert_yaxis()

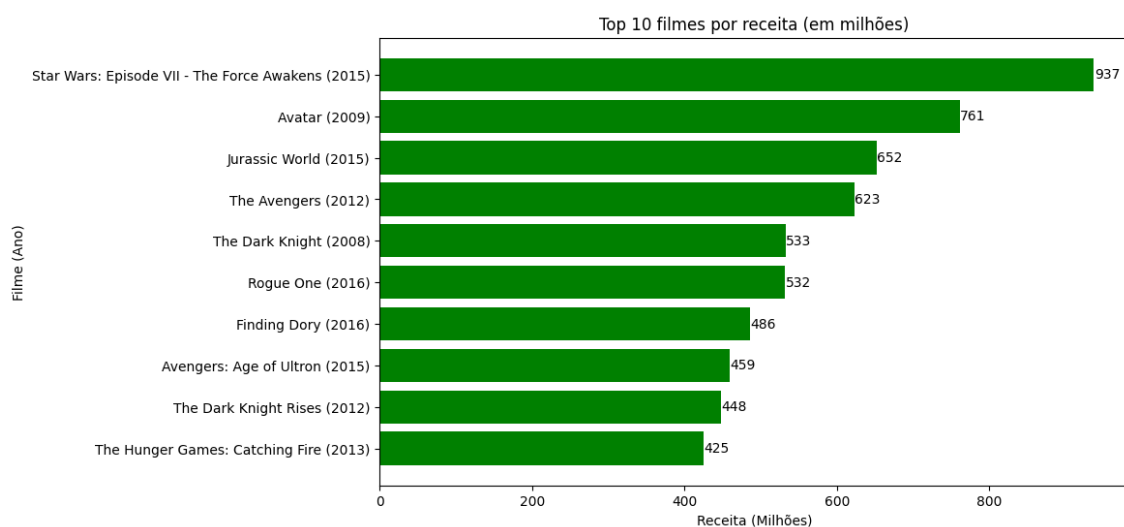
# valores nas barras
for bar in bars:
    width = bar.get_width()
    plt.text(
        width + 0.5,
        bar.get_y() + bar.get_height()/2,
        f"{width:.0f}",
        va="center"
    )

plt.show()
```

Foi utilizada uma visualização de barras horizontais para representar os 10 filmes com maior receita na base de dados. Esse tipo de gráfico é adequado para análises de ranking, pois permite uma comparação direta entre os maiores

valores, facilitando a identificação das produções de maior desempenho comercial.

A escolha por barras horizontais melhora a legibilidade dos nomes dos filmes, que podem ser extensos, além de facilitar a interpretação visual dos resultados. Adicionalmente, foi incluído o ano de lançamento de cada filme na visualização, proporcionando maior contextualização temporal e permitindo identificar em quais períodos ocorreram as produções de maior arrecadação da base analisada.



Agora analisando as pessoas envolvidas em cada filme, foi desenvolvido um gráfico com os diretores que mais dirigiram filmes na base de dados, juntamente com a receita média obtida entre suas produções. A proposta dessa análise foi verificar se existe proporcionalidade entre a quantidade de filmes dirigidos e o desempenho financeiro médio alcançado pelos diretores.

Durante o desenvolvimento, também foi realizada uma implementação complementar relacionada à nomenclatura exibida no gráfico de pizza, permitindo adicionar informações adicionais diretamente nos rótulos da visualização e tornando a interpretação dos resultados mais clara e informativa. Ajuda pontual com a nomenclatura exibida no gráfico de pizza.

```
top_directors = (  
    df.groupby("Director")  
    .size()
```

```

        .reset_index(name="movies_count")
        .sort_values("movies_count", ascending=False)
        .head(5)
    )

    director_revenue = (
        df.groupby("Director")["RevenueMillions"]
        .mean()
        .reset_index(name="avg_revenue")
    )

    top_directors = top_directors.merge(director_revenue, on="Director",
    how="left")
    top_directors["label"] = (
        top_directors["Director"] +
        "\nAvg: $" +
        top_directors["avg_revenue"].round(2).astype(str) +
        "M"
    )

    plt.figure(figsize=(8,8))

    plt.pie(
        top_directors["movies_count"],
        labels=top_directors["label"],
        autopct=lambda p: f'{int(round(p *
top_directors["movies_count"].sum() / 100))}'
    )

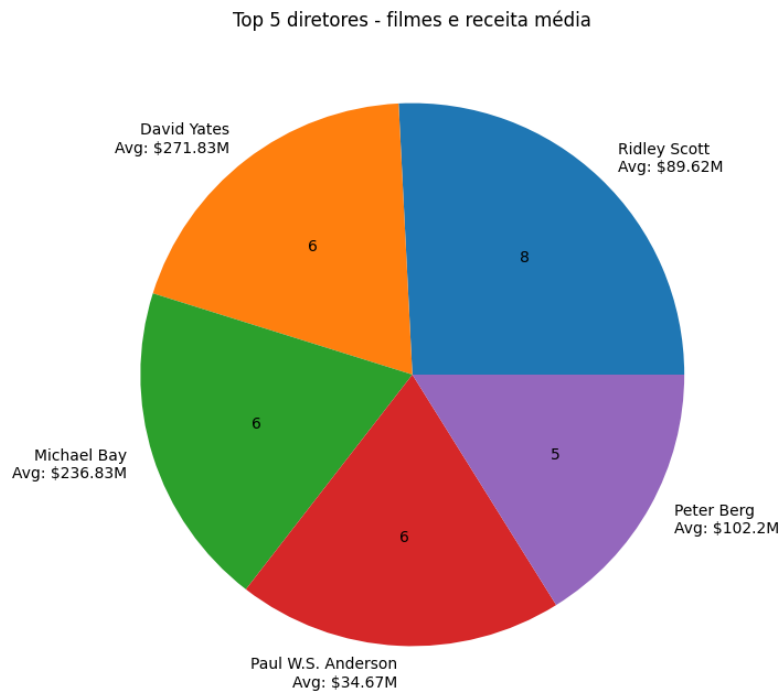
    plt.title("Top 5 diretores - filmes e receita média")
    plt.show()

```

Foi utilizado um gráfico de pizza para representar a participação dos diretores com maior quantidade de filmes produzidos na base de dados. Essa visualização foi escolhida por permitir uma interpretação rápida da distribuição proporcional entre os principais diretores, destacando quais possuem maior representatividade no conjunto analisado.

Além da quantidade de filmes, foram adicionadas informações de receita média dos filmes de cada diretor, enriquecendo a análise ao combinar produtividade e desempenho financeiro em uma única visualização. Dessa forma, é possível identificar não apenas quais diretores produziram mais filmes, mas também comparar o retorno médio obtido por suas produções.

A utilização do gráfico de pizza é adequada nesse contexto devido ao número reduzido de categorias analisadas (Top 5), o que mantém a leitura visual clara e facilita a percepção das diferenças de participação entre os diretores selecionados.



Após isso, foi realizada uma análise dos gêneros de filmes mais presentes na base de dados, juntamente com a receita média associada a cada gênero. Essa abordagem permitiu observar que uma maior quantidade de produções não necessariamente está relacionada a uma receita média mais elevada, além de possibilitar a identificação dos gêneros com maior potencial comercial e preferência do público.

Durante o desenvolvimento, também foi realizada uma implementação complementar relacionada aos nomes exibidos no eixo Y do gráfico, permitindo apresentar simultaneamente o gênero e sua respectiva receita média, enriquecendo a visualização e tornando a interpretação das informações mais intuitiva. Ajuda pontual com o nome exibido no eixo y, para apresentar a receita também.

```
df["Genre"] = df["Genre"].astype(str).str.strip()  
df["Genre"] = df["Genre"].str.split(r"\s*,\s*")
```

```

df_exploded = df.explode("Genre")
df_exploded["Genre"] = df_exploded["Genre"].str.strip()
#display(df_exploded)

genre_count = (
    df_exploded["Genre"]
    .value_counts()
    .reset_index()
)

genre_count.columns = ["Genre", "Count"]
#display(genre_count)

genre_revenue = (
    df_exploded.groupby("Genre")["RevenueMillions"]
    .mean()
    .reset_index(name="Avg_Revenue")
)

genre_summary = genre_count.merge(
    genre_revenue,
    on="Genre",
    how="left"
)

top_genres = genre_summary.head(10).copy()

top_genres["label"] = (
    top_genres["Genre"] +
    " | Avg: $" +
    top_genres["Avg_Revenue"].round(1).astype(str) +
    "M"
)

plt.figure(figsize=(12,6))

bars = plt.barh(
    top_genres["label"],
    top_genres["Count"],
    color="g"
)

plt.title("Top 10 gêneros mais produzidos e receita média")
plt.xlabel("Quantidade de filmes")
plt.ylabel("Gênero")
plt.gca().invert_yaxis()
for bar in bars:
    width = bar.get_width()
    plt.text(
        width + 0.5,

```

```

        bar.get_y() + bar.get_height()/2,
        int(width),
        va="center"
    )

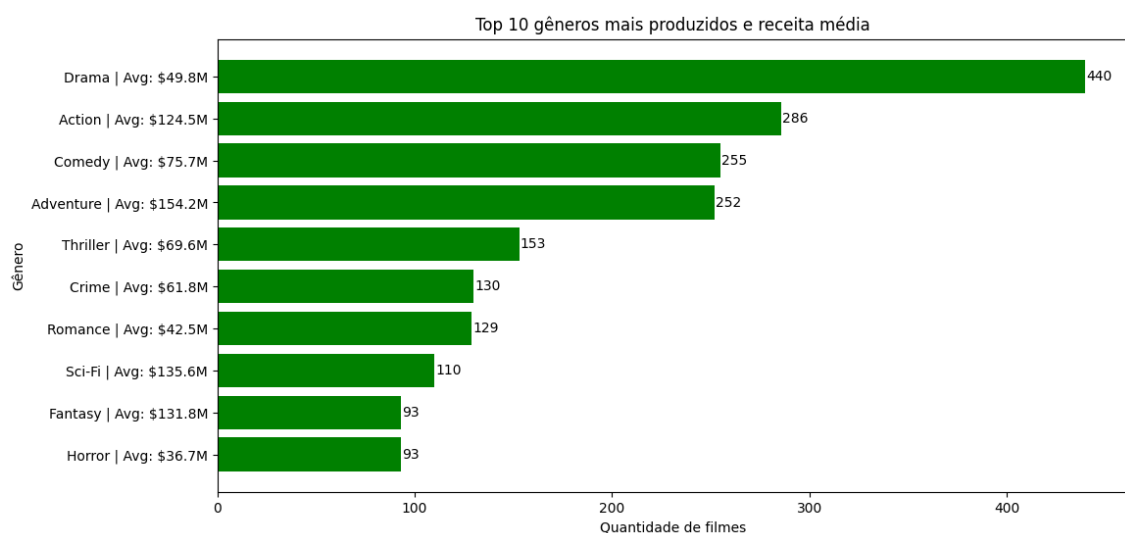
plt.show()

```

Foi utilizado um gráfico de barras horizontais para representar os gêneros cinematográficos mais produzidos na base de dados. Esse tipo de visualização é especialmente adequado para análises de ranking, pois facilita a comparação direta entre categorias e melhora a leitura de rótulos textuais maiores, como os nomes dos gêneros.

Além da quantidade de filmes produzidos, foi incorporada a receita média dos filmes de cada gênero, permitindo uma análise complementar entre volume de produção e desempenho financeiro médio. Essa abordagem possibilita identificar, simultaneamente, quais gêneros possuem maior presença na base e quais apresentam maior potencial de receita.

A escolha por barras horizontais também favorece a visualização ordenada dos dados, destacando de forma intuitiva os gêneros mais relevantes e tornando a interpretação dos resultados mais clara e objetiva.



Após isso, foi desenvolvida uma análise para identificar quais atores apresentaram maior participação em filmes durante o período analisado da base

de dados, evidenciando aqueles com maior presença nas produções cinematográficas. Além da quantidade de participações, também foi analisada a receita média dos filmes em que esses atores estiveram envolvidos, permitindo comparar frequência de atuação e desempenho financeiro médio das produções associadas a cada profissional.

```
df["Actors"] = df["Actors"].astype(str).str.strip()
df["Actors"] = df["Actors"].str.split(r"\s*,\s*")
df_exploded = df.explode("Actors")
df_exploded["Actors"] = df_exploded["Actors"].str.strip()
#display(df_exploded)

actors_count = (
    df_exploded["Actors"]
    .value_counts()
    .reset_index()
)

actors_count.columns = ["Actors", "Count"]

actors_revenue = (
    df_exploded.groupby("Actors")["RevenueMillions"]
    .mean()
    .reset_index(name="Avg_Revenue")
)

actors_summary = actors_count.merge(
    actors_revenue,
    on="Actors",
    how="left"
)

top_actors = actors_summary.head(10).copy()
top_actors["label"] = (
    top_actors["Actors"] +
    " | Avg: $" +
    top_actors["Avg_Revenue"].round(1).astype(str) +
    "M"
)

plt.figure(figsize=(12,6))

bars = plt.barh(
    top_actors["label"],
    top_actors["Count"],
    color="g"
)
```

```
plt.title("Top 10 atores com maior participação em filmes")
plt.xlabel("Quantidade de filmes")
plt.ylabel("Atores")

plt.gca().invert_yaxis()
for bar in bars:
    width = bar.get_width()

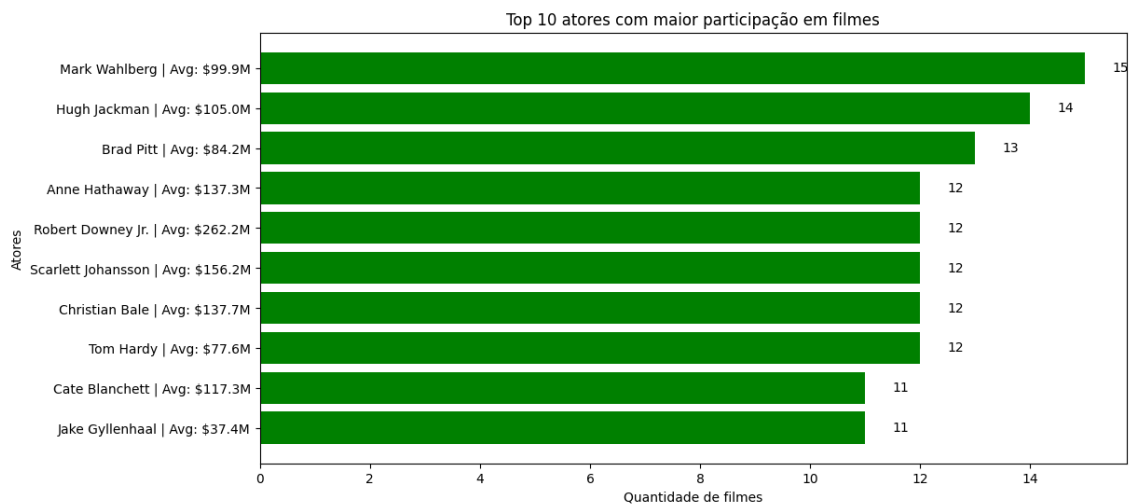
    plt.text(
        width + 0.5,
        bar.get_y() + bar.get_height()/2,
        int(width),
        va="center"
    )

plt.show()
```

Foi utilizado um gráfico de barras horizontais para representar os atores com maior participação em filmes na base de dados. Esse tipo de visualização é adequado para análises de ranking, pois permite comparar de forma clara e objetiva a quantidade de filmes associados a cada ator, além de facilitar a leitura de nomes maiores sem comprometer a interpretação visual.

Além da contagem de participações, foi incorporada ao gráfico a receita média dos filmes em que cada ator esteve presente, enriquecendo a análise ao combinar frequência de participação com desempenho financeiro médio das produções. Dessa forma, a visualização não apenas identifica os atores mais recorrentes da base, mas também permite observar o potencial comercial médio associado às suas filmografias.

A utilização de barras horizontais favorece a organização dos dados em ordem decrescente, destacando visualmente os atores com maior representatividade e tornando a comparação entre categorias mais intuitiva e eficiente.



E, para finalizar, foi realizada uma análise do tempo médio de duração dos filmes ao longo dos anos, com o objetivo de compreender como a indústria cinematográfica tem se comportado ao decorrer do tempo em relação ao perfil das produções lançadas. Essa análise permite identificar tendências de aumento ou redução na duração média dos filmes, auxiliando na interpretação de possíveis mudanças no padrão de consumo do público e nas estratégias adotadas pela indústria cinematográfica.

```
df["Runtime"] = pd.to_numeric(df["Runtime"], errors="coerce")
df = df.dropna()
df_year = df.groupby("Year")["Runtime"].mean().reset_index()
df_year = df_year.sort_values("Year")

plt.figure(figsize=(10,6))

plt.plot(df_year["Year"], df_year["Runtime"], marker="o", color="g")

plt.title("Runtime médio dos filmes por ano")
plt.xlabel("Ano")
plt.ylabel("Duração média (minutos)")

plt.grid(True)

plt.show()
```

Foi utilizada uma visualização de linha para analisar a evolução do tempo médio de duração dos filmes ao longo dos anos. Esse tipo de gráfico é adequado para identificar tendências temporais, permitindo observar se há aumento ou redução

na duração média das produções cinematográficas ao longo do período analisado.

A agregação por ano permite reduzir variações individuais e destacar o comportamento geral da indústria ao longo do tempo, facilitando a interpretação de mudanças estruturais no padrão de produção dos filmes.

